CPU ARCHITECTURE DESIGN

Contents

Introduction 2

Category-based addressing & Execution 3

CPU Instruction Set 4

CPU Architecture Branch Handling and Priority System 5

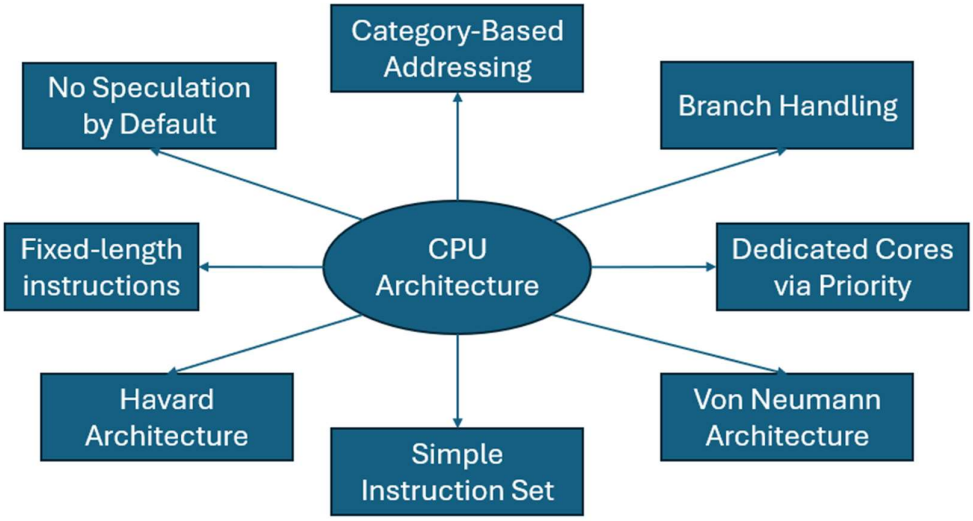
CPU Workflow 6

CPU Architecture Trade-Offs 7

Introduction

This CPU Architecture is designed to improve the performance and security of CPUs. This architecture was also designed to address the current issues and limitations of the CPU. The CPU architecture was developed as currently CPUs aren't fit and designed to handle modern computer workloads. This was also developed to fit in and ensure full compatibility with my other designs. This CPU architecture will cover the FDE cycle, instruction set and branching handling. It will also show a workflow of what the CPU could look like in action along with trade-offs of this CPU architecture.

This CPU Architecture cannot promise perfect performance or security at any point during usage. However, this CPU architecture can improve efficiency, security and performance of CPUs across devices. This CPU architecture is also compatible with the Von Neumann architecture and Harvard architecture. This is just a design that engineers can use as reference and guide for implementing this new CPU architecture. Make sure to adapt features and systems based on resources and budget.



The spider diagram shows some of the key features of this CPU Architecture. We can clearly see that it is compatible with both current computing architectures. This spider diagram does not show all features involved in the CPU architecture. More details can be found in the sections below.

Category-based addressing & Execution

- **Category-based addressing is where the data and instructions stored in cache and RAM are sorted into categories.**
- **Category-based addressing allows for the CPU to fetch instructions from memory more quickly leading to quicker execution.**
- **There are 6 components within the CPU that category-based addressing affects.**
- **The first component is the program counter otherwise known as PC.**
- **There will need to be 3 program counters in total to support parallel fetching across all 3 categories.**
- **For example, OS PC would show OS: 0x1000, App PC would show App: 0x00003A00 and BG PC would show BG: 0x00001F2A.**
- **This change is needed because it allows the CPU to coordinate the 3 address buses correctly for parallel fetching.**
- **The second component is the memory address register.**
- **There would need to be 3 memory address registers to store the addresses for all 3 category instructions at once until the address buses are available.**
- **This change is needed because it allows the CPU's address buses to gain the addresses for all 3 categories at once.**
- **The third component is the address bus.**
- **There would need to be 3 address buses to find the instruction in each category at the same time.**
- **These address buses would be 64-bit in length and would be configured to go to a specific MAR to gain the address for the correct category.**
- **This change is needed because it allows the CPU to find the instructions in all 3 categories at once.**
- **The fourth component is the memory data register.**
- **There would need to be 3 memory data registers to store data and instructions from each category at once.**
- **There is no need for 6 MDR's as they can hold both data and instructions allowing them to manage both from each category.**
- **This change is needed because it allows the CPU to store both data and instructions from all 3 categories at once.**
- **The fifth component is the CIR**
- **There would need to be 3 CIRs to allow the CPU to execute and decode instructions from all 3 categories at once.**
- **This change is important as it allows the CPU to perform parallel decoding and execution.**
- **The control unit would need to be configured to manage the data flow between categories and allocate CPU time per process within categories.**
- **This change is needed because it allows the CPU to avoid cross-category issues and make sure that more important processes get more CPU time.**

CPU Instruction Set

- **An instruction set is how instructions are organized and formatted within memory before the CPU decodes them.**
- **This CPU architecture changes the instruction set compared to the x86 which currently uses variable length instructions.**
- **Fixed-length instructions are used in this architecture to provide more predictable performance and avoid complexity in decoding**
- **This means that the CPU can decode faster as it does not need to figure out the length of the instruction before splitting into opcode and operand.**
- **These fixed-length instructions can be 32-bits or 64-bits in length with a simple opcode and operand split so the CPU does not need to perform guesswork.**
- **For 32-bit instructions, we would have 8 bits for opcode and 24 bits for operand.**
- **For 64-bit instructions, we would have 16 bits for opcode and 48 bits for operand.**
- **This improves performance as the CPU can decode instructions quickly overall leading to faster execution since instructions are understood more quickly.**
- **Fixed-length instructions will require changes to the complier as it is changing from traditional instruction sets.**
- **The complier requires changes because the complier needs to differentiate between categories and ensure that instructions are fixed length.**
- **First, the complier would need to be aware of which category it is dealing with to store the instructions in the correct place.**
- **The complier will find this out through tags attached to the file as the OS will tag each file according to the partition it is stored within.**
- **Second, the complier ensures that the code is translated to machine code and into the opcode and operand split.**
- **The complier does this by first translating into machine code and then the fixed length for the opcode and operand split.**
- **The DMA and CPU manages the task of making sure the instructions are stored within the correct category using the tag system and other methods.**
- **These changes are needed to ensure that the CPU has readable instructions and that it can perform the opcode and operand split correctly.**
- **OS's memory mapping will also require changes to ensure that instructions don't mix between categories.**
- **First, the OS will dedicate specific address ranges for each RAM slot/category to prevent confusion.**
- **These specific address ranges will not change to avoid the confusion of using different address ranges each time for each category.**
- **These changes are important to ensure that the instructions are placed within the categories to avoid the CPU wasting cycles in not being able to find instructions.**
- **You can look at my RAM architecture design to find out more about OS memory management within categories.**

CPU Architecture Branch Handling and Priority System

- **The CPU currently uses branch handling to guess what the path of the next program is going to be.**
- **There are two options to branch handling that this CPU Architecture needs.**
- **The first option is no speculation by default.**
- **This means that the CPU must fetch data early before the CPU branching reaches the final breach.**
- **This then leads to the data arriving on time due to the speed of category-based addressing and allows the CPU to make the correct decision.**
- **However, this option may not be compatible with my CPU architecture as its fetching process can be quicker than branching.**
- **The second option is no branching handling.**
- **This means that the CPU doesn't use branching as it can fetch and decode instructions extremely quickly meaning less wait time.**
- **This then leads to no mispredictions meaning security vulnerabilities like Spectre and Meltdown are eliminated completely.**
- **This CPU architecture also uses a priority system for dedicating cores and CPU time.**
- **There are two types of priority systems that this CPU Architecture can use.**
- **The first type is dynamic priority.**
- **This is where the CPU dedicates core for each category and decides which category gets the most CPU time based on workload.**
- **This can be done through the CU as it can manage CPU time allocation in real-time while the operating system handles the core allocation.**
- **However, this dynamic priority can have latency issues as the cores must change categories depending on workload which may reduce performance.**
- **This dynamic priority improves adaptability and performance overall as it allows cores to focus on one category and allows CPU time to be managed in real-time.**
- **The second type is fixed priority.**
- **This is where the CPU dedicates cores for each category once and allocates CPU time fixedly with OS category processes getting the most CPU time.**
- **This can be done also through the CU as it can manage the CPU time allocation for each category, and the operating system can create fixed dedication for each core.**
- **However, this fixed priority can mean that the performance of applications or the operating system can slower as only one gains the most priority throughout CPU lifespan.**
- **This fixed priority improves performance as it allows cores to focus on one category and allocate CPU time where its commonly needed without latency issues like the dynamic priority can experience.**

CPU Workflow

- **This CPU workflow will cover the process of CPU setup and FDE cycle during active use of a device.**
- **First, the CPU powers on for the first time and immediately fetches, decodes and executes instructions from RAM and internal drive to load the operating system.**
- **Second, the CU within the CPU gives the most CPU time to OS category processes on the device.**
- **This CPU uses the dynamic priority system which is currently assigning the most CPU time to operating system processes as no applications have been used.**
- **Third, the operating system dedicates the most CPU cores to OS category processes as currently the workload is the heaviest there.**
- **Fourth, the user installs GTA 6 and starts the application on their device.**
- **Fifth, the operating system allocates the most CPU cores to the application category processes as it is currently the heaviest workload in the system.**
- **Sixth, the CU gives the most CPU time to App category processes on the device.**
- **Seventh, the CPU repeatedly does the FDE cycle for instructions on the device.**
- **The CPU executes the 3 FDE cycles repeatedly throughout usage of the device**
- **The program counters show instructions for all 3 categories which is App: 0x3A00, OS: 0x00007C1E and BG: 0x00001F2A.**
- **The memory address registers temporarily store the addresses from all 3 categories.**
- **The address buses travels to cache checking levels based on the categories.**
- **The OS address bus checks Level 1 cache, the App address bus checks Level 2 cache and the BG bus check Level 3 cache.**
- **The address buses then navigate to their defined categories to search for the instructions in memory.**
- **The data bus and instruction bus for each category then transport the data to the 3 memory data registers which stores it temporarily.**
- **The 3 CIRs then hold the 3 instructions while the CPU's cores decode the instructions and execute them depending on the instruction.**
- **The CPU splits each instruction into opcode and operand based on the split it is told to expect.**
- **The CPU executes these 3 instructions by using the ALU, storing the result in the accumulator or using other components within the device.**
- **The results are then written directly back to the category depending on what the instruction was.**
- **All 3 program counters increment by 1 and the next instructions within memory are shown repeating the FDE cycle once again.**

CPU Architecture Trade-Offs

- **This CPU Architecture has trade-offs just like any other system I have created so far.**
- **There also may be additional implementation challenges that developers and engineers may face.**
- **I have identified 6 main trade-offs with this CPU Architecture.**
- **The first trade-off is heat.**
- **This trade-off exists because since the CPU can complete the FDE cycle more quickly and efficiently, it can also produce more heat.**
- **This means that the CPU may overheat if it works too quickly without limitations.**
- **This trade-off can be managed by cooling systems, clock speed limitations and heat monitoring to ensure the CPU doesn't overheat.**
- **The second trade-off is performance.**
- **This trade-off exists because dynamic priority can have latency delays and the Von Neumann architecture may be less effective due to the bottleneck of the bus.**
- **This trade-off can be managed by using Havard Architecture where possible, rate-limiting to ensure that cores do not change too frequently and accepting the limitations of the Von Neumann architecture.**
- **Thie third trade-off is cost.**
- **This trade-off exists because developing this CPU architecture requires coordination between OS vendors and hardware developers as well as materials and budget to build.**
- **This trade-off be managed by using pre-existing agreements, budget management and testing the CPU architecture before full development.**
- **The fourth trade-off is time-consumption.**
- **This trade-off exists because developing this CPU architecture will take years due to the software and firmware changes required.**
- **This trade-off can be managed by structured development processes, role-based work and strong coordination between OS and hardware vendors.**
- **The fifth trade-off is security**
- **This trade-off exists because if this CPU architecture were to include branch handling, then security vulnerabilities like Spectre and Meltdown can be exploited.**
- **This trade-off can be managed by implementing the CPU architecture without the branching or accepting the risk of exploitation.**
- **The sixth trade-off is lack of legacy support**
- **This trade-off exists as this CPU architecture will mostly likely require a new generation of CPU.**
- **This trade-off can be managed by implementing the category-awareness through firmware/software updates for current devices and accepting that it won't be compatible for everyone.**